

Nottingham Trent University

Scale: Deep Reinforcement Learning for Container Scheduling in Serverless Edge Computing

Chen Chen (Nottingham Trent University)

Background



What is serverless ?

A computing paradigm:

- Function developers only focus on the function code
- Resource management and configuration are managed by the platform
- Pay-as-you-use billing model
- Users are charged based on resource usage, number of requests and duration
- Serverless functions are only invoked by events/requests
- Functions are terminated after some time when they finish execution
- Lightweight and elastic



Serverless Stack





Applications







Engines





Distributed Frameworks
(run distributed apps)

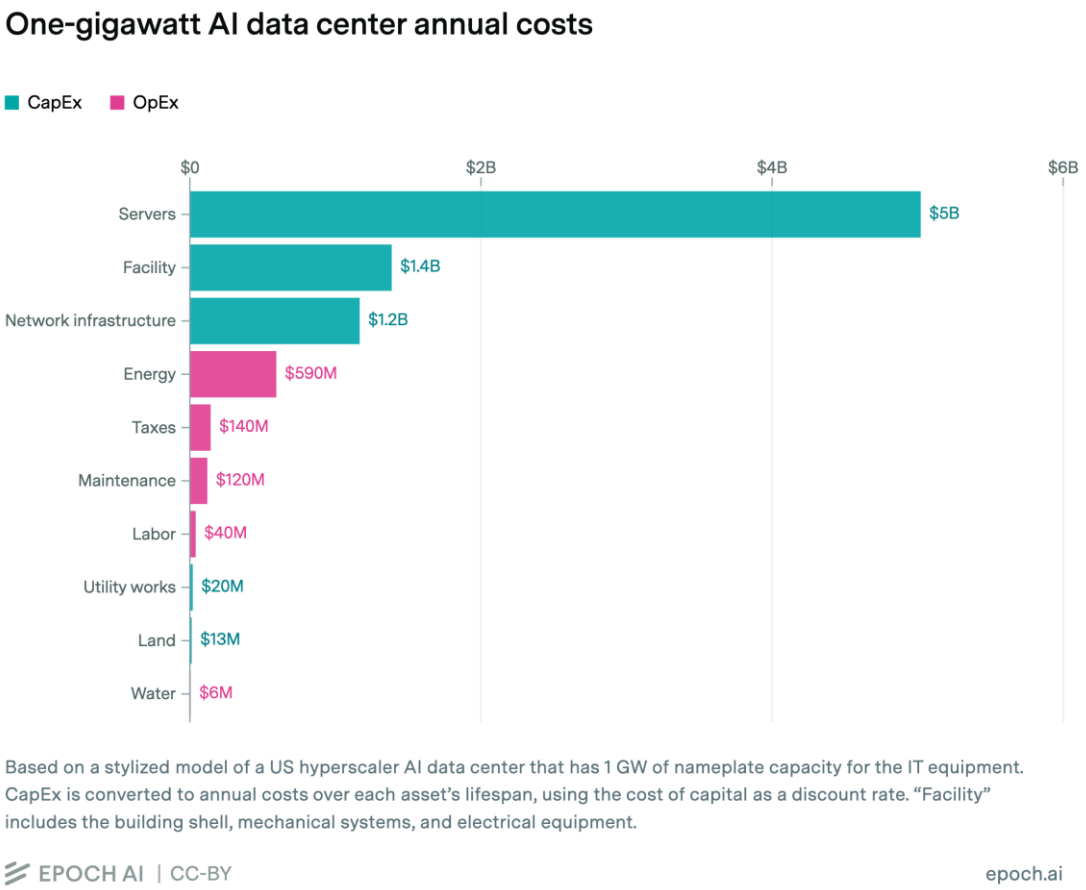




Runtime Layer
(memory allocation/libraries)

NTU

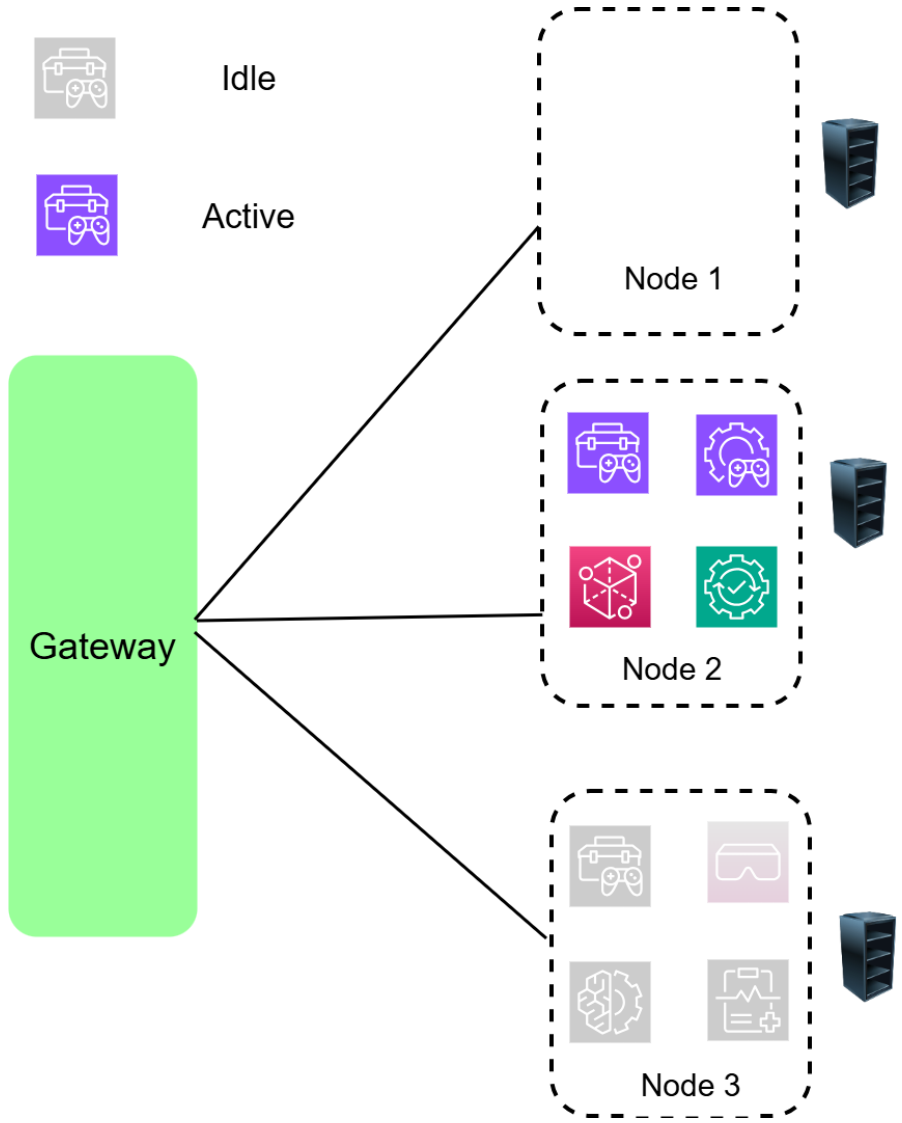
Servers account for 60% of the total cost of AI data center



https://epoch.ai/data-insights/ai-datacenter-cost-breakdown

Motivation

- Node selection
- Create a new function or use an existing function?
- Cache or kill?



Motivation

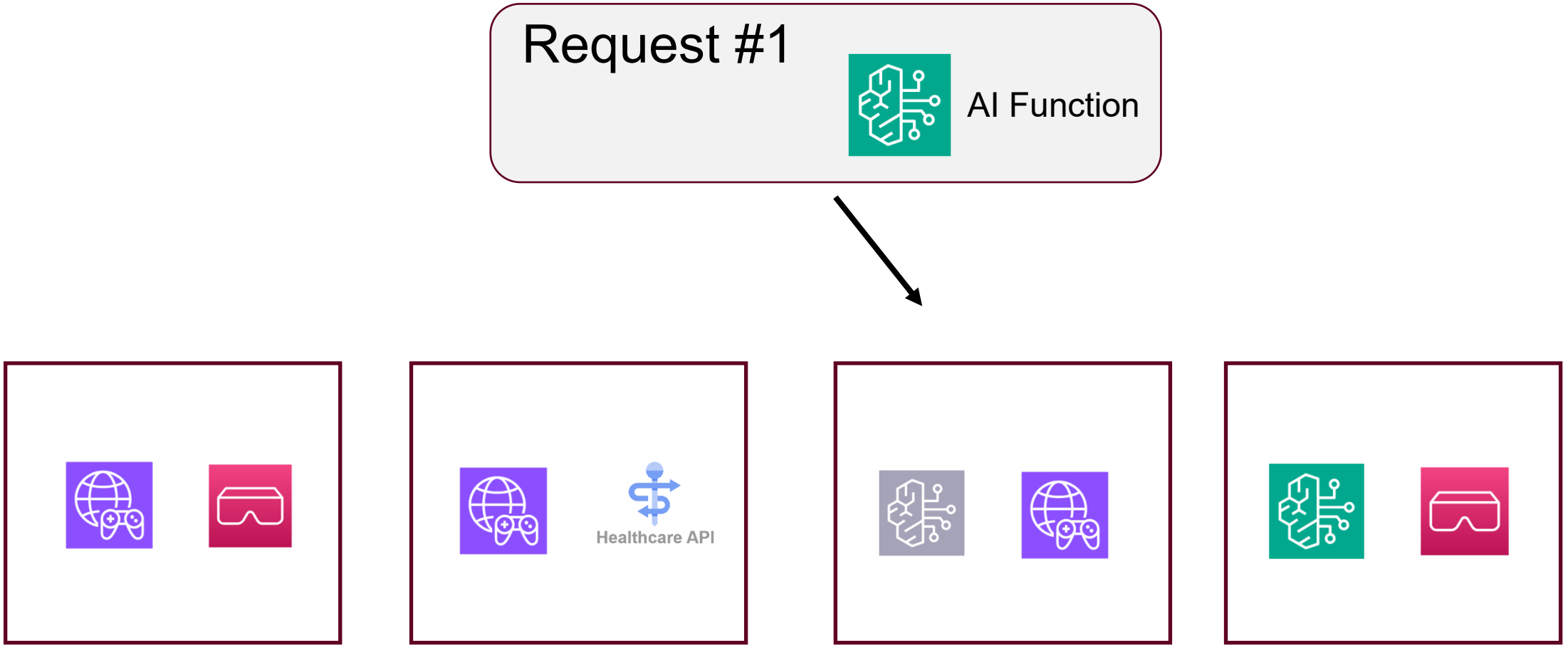
Existing works mostly focus on placement or caching for:

- End-to-end latency optimization
- Deployment cost optimization
- Energy consumption or carbon emission
- Startup process of serverless functions

Therefore, we decided to consider both placement and caching while guaranteeing the SLO for users

NTU

Motivation


Request #1
AI Function
Healthcare API
Node 1
Node 2
Node 3
Node 4

Problem formulation



Problem formulation

Objective: **E2E latency for serverless functions**

min: Coldstart latency+ Computation latency + Communication latency

$$\min T = \sum_{k=K} \left(T_{cold}^{k} + T_{comp}^{k} + T_{comm}^{K} \right)$$

Constraints:

- Edge node capacity (e.g., CPU and memory)
- SLO for each request
- Integer constraint for decision variables

Problem formulation

- Reduced from the Generalized Assignment Problem, which is NP-hard.
- To handle the computational complexity, we chose Actor-Critic-based DRL.
- We have selected Proximal Policy Optimization (PPO).

NTU

The idea



DRL workflow



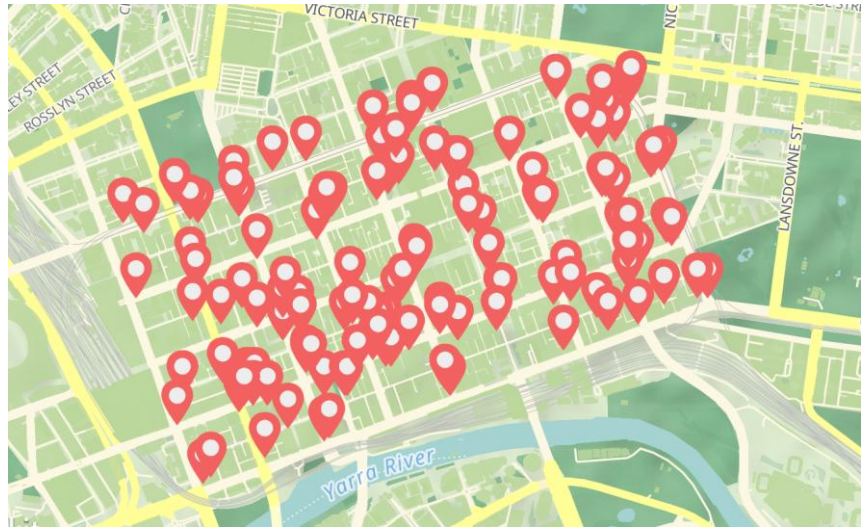
$$r(t) = -\left(T_{cold} + T_{comp} + T_{comm}\right)$$

Performance



Simulation setup

- Topology – EUA dataset[1], 125 edge nodes
- Requests – Huawei serverless dataset[2]
- DRL – Stable baseline 3[3]
- Baseline – m-DQN, Midaco solver



[1] https://github.com/PhuLai/eua-dataset
[2] https://github.com/sir-lab/data-release
[3] https://stable-baselines3.readthedocs.io/en/master/

Results

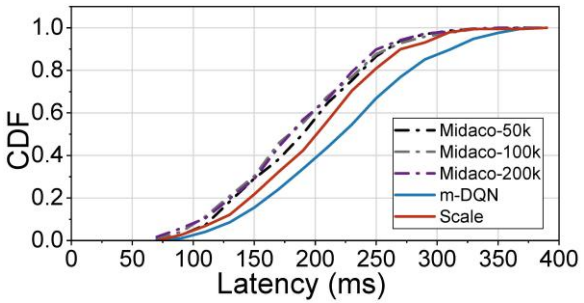


Fig. 1 CDF of latency

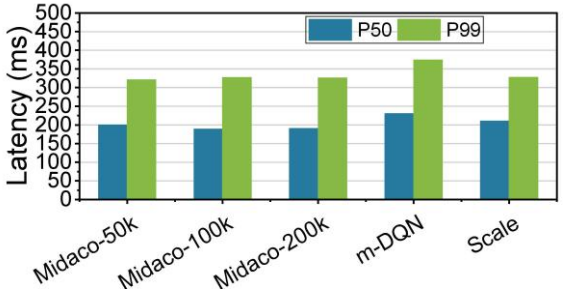


Fig. 2 P50 and P99

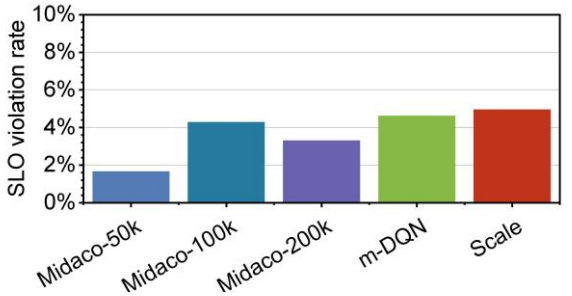


Fig. 3 SLO violations

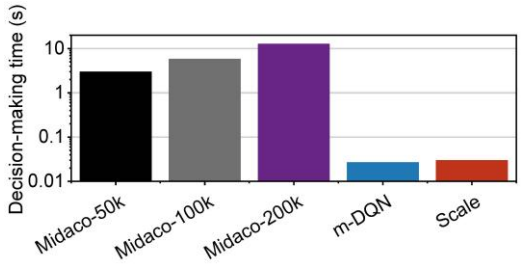


Fig. 4 Scheduling time

- Midaco only improves 3.08% from 50k rounds to 200k rounds, - approximating the optimum
- Scale – within a factor of 1.15 to Midaco
- Comparable P50 and P99 latency
- Scale receives comparable performance for SLO violations
- The decision-making time is 99% less than that of Midaco

NTU

Takeaway

- AI Demand and capital investment are growing fast
- The problem can be formulated as an ILP problem with an SLO constraint
- DRL can efficiently achieve near-optimal solutions while maintaining SLOs
- The scheduling time is significantly reduced

NTU



Nottingham Trent University

Thank you

chen.chen04@ntu.ac.uk